LAPORAN PRAKTIKUM 4
RELASI KELAS DAN ARRAY OBJEK



Disusun Oleh:

KHOHAR MUHAMAD FATAHURROHMAN

254107020021

Kelas TI 2B

PROGRAM STUDI TEKNIK INFORMATIKA

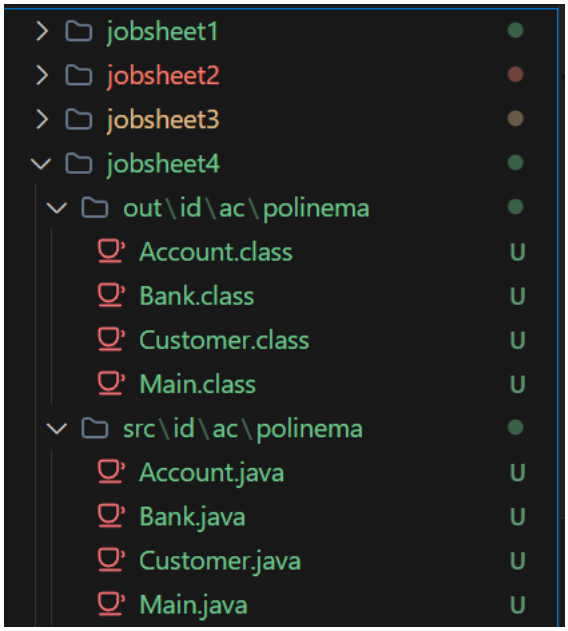
JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI MALANG

September 2026

1. Langkah Percobaan

a. Struktur Project



Penjelasan Struktur Project:

Proyek ini melanjutkan bank-mini dengan menambahkan konsep Relasi Antar Kelas. Di dalam package id.ac.polinema, sekarang terdapat 4 buah kelas utama:

1. Customer.java: Representasi nasabah (menyimpan nama dan nomor telepon).
2. Account.java: Representasi akun rekening (berelasi dengan Customer).
3. Bank.java: Representasi kumpulan akun yang disimpan dalam Array (Aggregation).
4. Main.java: Kelas untuk melakukan pengujian logika kode.

b. Implementasi & Menjalankan Program

[!] PASTE SCREENSHOT SOURCE CODE DI SINI

[!] PASTE SCREENSHOT OUTPUT PROGRAM (Terminal) DI SINI

Penjelasan Kode:

Pada Jobsheet 4 ini, terjadi perubahan penting di mana atribut ownerName bertipe String pada Account diubah menjadi objek Customer. Ini adalah bentuk relasi Association di mana Account 'memiliki/mengetahui' Customer-nya. Selain itu, dibuat class Bank yang bertugas menyimpan banyak referensi objek Account ke dalam sebuah array. Untuk melakukan pencarian, dibuat algoritma iterasi sederhana (looping) pada array untuk mencocokkan nomor rekening (findAccount) maupun mencocokkan nama Customer (findAccountsByOwnerName).

2. Hasil & Pembahasan

Dari output program yang dihasilkan, terbukti bahwa relasi antarkelas dan pengelolaan objek menggunakan array berjalan dengan mulus. Pembuatan objek Account kini mewajibkan pelemparan objek Customer ke dalamnya. Saat Bank memanggil fungsi findAccountsByOwnerName('Nadia'), program juga dengan tepat mengembalikan 2 buah akun terpisah yang dimiliki oleh orang yang sama, membuktikan bahwa satu entitas Customer dapat dikaitkan dengan banyak entitas Account.

3. Jawaban Tugas Mandiri

(a) Mengapa relasi Account-Customer disebut association, bukan aggregation atau composition?

Jawaban: Karena Account dan Customer adalah dua entitas yang saling independen dan masing-masing memiliki siklus hidup (lifecycle) sendiri di memori. Seorang Customer tetap bisa ada di sistem walaupun belum memiliki Account, dan sebuah Account hanya sekadar 'menyimpan referensi' ke Customer. Keduanya tidak terikat kuat layaknya induk-anak (Composition).

(b) Bayangkan objek Bank dihapus dari memori. Apakah objek Account yang pernah ditambahkan ke dalamnya ikut terhapus?

Jawaban: Objek Account seharusnya tetap bisa berdiri sendiri (tidak ikut terhapus) walaupun objek Bank dihancurkan dari memori. Hal ini karena array pada Bank hanya menyimpan 'alamat memori' (referensi) dari Account. Inilah inti dari relasi Aggregation (has-a yang lemah), di mana komponen penyusun (Account) tidak hancur saat wadahnya (Bank) hancur.

4. Kesimpulan

Kesimpulan dari praktikum ini:

1. Object Oriented Programming (OOP) memungkinkan kita memodelkan relasi dunia nyata dengan mudah, misalnya merelasikan class Customer dengan class Account.
2. Array tidak hanya berguna untuk menampung tipe data primitif, tapi sangat powerful untuk menampung referensi objek dalam jumlah banyak (seperti Array of Account di dalam class Bank).
3. Penggunaan relasi Aggregation dan Association memperjelas batasan tanggung jawab antarkelas tanpa harus meleburkan semuanya ke dalam satu class yang rumit.

5. Lampiran

Link Repositori GitHub untuk Source Code Lengkap:

https://github.com/helfa25/PRAKTIKUM-PBO.git